1 Gramática

```
Prog        → MainC DefCl

MainC       → 'class' Id '{'
              'public' 'static' 'void' 'main' '(' 'String' '[' ']' Id ')' '{' Cmd '}' '}'

DefCl       → 'class' Id DefCl'
              | λ

DefCl'      → '{' DefVar DefMet '}' DefCl
              | 'extends' Id '{' DefVar DefMet '}' DefCl

DefVar      → Type Id ';' DefVar
              | λ

DefMet      → 'public' Type Id '(' DefMetArgs ')'
              '{' DefVar Cmd 'return' Exp ';' '}' DefMet
              | λ

DefMetArgs  → Args
              | λ

Type        → 'int' '[' ']'
              | 'boolean'
              | 'int'
              | Id

Args        → Type Id Args'

Args'       → ',' Args
              | λ

Cmd         → '{' CmdList '}'
              | 'if' '(' Exp ')' Cmd 'else' Cmd
              | 'while' '(' Exp ')' Cmd
              | 'System' '.' 'out' '.' 'println' '(' Exp ')' ';'
              | Id Cmd'

Cmd'        → '=' Exp ';'
              | '[' Exp ']' '=' Exp ';'

CmdList     → Cmd CmdList
              | λ

Exp         → PrimExp Exp'
```

```
Exp'        → Op PrimExp Exp'
              | '[' Exp ']' Exp'
              | '.' PostExpDot
              | λ

Op          → '&&'
              | '>'
              | '+'
              | '-'
              | '*'

PostExpDot  → 'length' Exp'
              | Id '(' ListExp ')' Exp'

PrimExp     → 'new' PrimExp'
              | '!' Exp
              | '(' Exp ')'
              | 'true'
              | 'false'
              | Id
              | Number
              | 'this'

PrimExp'    → Id '(' ')'
              | 'int' '[' Exp ']'

Id          → Char Word

Word        → Char Word
              | Number Word
              | '_' Word
              | λ

Char        → 'a' | ... | 'z' | 'A' | ... | 'Z'

Number      → Digit Number'

Number'     → Digit Number'
              | λ

Digit       → '0' | '1' | '2' | '3' | '4'
              | '5' | '6' | '7' | '8' | '9'
```